PAYROLL MANAGEMENT SYSTEM

PROJECT SYNOPSIS

OF MINOR PROJECT

BACHELOR OF TECHNOLOGY

(Information Technology)

SUBMITTED BY:

Jasdeep Singh (Roll No. 2435556)

Jashanpreet Kaur (Roll No. 2435558)

Jiya Sharma (Roll No. 2435566)

Jashanjot Singh (Roll No. 2435557)

Jasmeet Nain(Roll No. 2435561)

SUBMITTED TO:

Dr.Kulvinder Singh Mann

(Prof. & Head IT Deptt.)

JUNE-JULY, 2025-26

GURU NANAK DEV ENGINEERING COLLEGE, LUDHIANA

(An Autonomous College under UGC ACT)

Introduction

1. Need of Payroll Management System

If you've ever worked in an office or even managed a small team, you probably know how messy salary management can get. Keeping track of who worked how many days, calculating overtime, handling tax deductions, managing leaves — it sounds simple on paper, but in practice it can quickly become overwhelming. Most small organizations still rely on spreadsheets or even handwritten registers for this, which works fine initially but starts falling apart as the team grows.

This is exactly why a Payroll Management System is needed. Instead of someone sitting down every month and manually calculating each employee's salary — cross-checking attendance, applying tax rules, adjusting for bonuses or unpaid leaves — a system can do all of this automatically, accurately, and in far less time.

Beyond just saving time, there's also the issue of errors. A single wrong formula in a spreadsheet can affect everyone's salary that month, and correcting it is both embarrassing and time-consuming. Employees losing trust in how their salary is calculated is a serious problem for any organization.

2. Why Payroll Management System is Required

Let's be honest, nobody enjoys doing repetitive calculations month after month, especially when a single mistake can cause real problems. If an employee gets paid less than they should, they're going to be frustrated. If salary gets delayed because someone was manually cross-checking hundreds of entries, morale takes a hit. These aren't hypothetical situations they happen regularly in organizations that still rely on manual payroll processes.

This is where a Payroll Management System actually makes a difference. Instead of someone spending hours calculating gross salary, figuring out tax deductions, adjusting for leaves and overtime, and then generating individual pay slips the system handles all of it automatically. What used to take days can be done in minutes, with far fewer chances of something going wrong.

Research also backs this up. Studies in the field of Human Resource Information Systems have shown that automating payroll can reduce administrative workload by around 30–40% in small and medium-sized organizations (Beadles et al., 2005). Kovach & Cathcart (1999) similarly found that computerized HR systems lead to better record-keeping and overall organizational effectiveness. So this isn't just a convenience there's solid evidence that it genuinely improves how organizations function.

Data security is another reason this system matters. Salary details, bank information, tax records, this is sensitive data that shouldn't be sitting in an unlocked spreadsheet that anyone on the team can open. A proper system with login authentication and role-based access makes sure only the right people can view or edit this information.

So really, a Payroll Management System isn't just about calculating salaries faster. It's about making the whole process more reliable, more secure, and less stressful for everyone involved.

3. Methods Required for the Payroll Management System

First, the system will follow the System Development Life Cycle (SDLC) methodology. This includes requirement analysis, system design, implementation, testing, and deployment. During requirement analysis, we identify what features the system must include — such as employee registration, attendance management, salary calculation module, tax deduction module, report generation, and pay slip generation.

Second, a database-driven approach can be used. A relational database (like MySQL or SQL Server) can store employee details, attendance records, salary structure, and transaction history. Proper normalization techniques can ensure data consistency and reduce redundancy.

Third, the payroll calculation method will be based on predefined formulas:

Gross Salary = Basic Salary + Allowances

Net Salary = Gross Salary – (Tax + Deductions)

These formulas will be implemented programmatically to ensure automatic computation. Conditional logic will be applied to handle overtime, bonuses, unpaid leaves, and tax slabs.

Fourth, the system can use authentication and authorization mechanisms. Admin access can be separate from employee access. Role-based login ensures security and controlled data access.

Additionally, research in payroll automation systems suggests integrating modular design practices. A modular approach increases maintainability and scalability of the system (Kavanagh & Johnson, 2017, Human Resource Information Systems). This means each module — employee management, salary processing, reporting — can function independently yet remain connected within the system.

If needed, Agile methodology can also be partially adopted for incremental development, especially since this is a mini project with time constraints.

Literature View

1. Beadles et al. (2005)

Title: The Impact of Human Resource Information Systems: An Explorary Study in the Public Sector

This paper was one of the first we came across, and it was actually quite relevant. Beadles and his team looked at how automating HR functions, including payroll, affected organizations in the public sector. Their main finding was that organizations using automated systems made significantly fewer errors and managed data much better than those doing things manually. For us, this was a good starting point because it confirmed that even a simple automated payroll system is genuinely better than doing things by hand. The paper is more about HRIS in general rather than payroll specifically, but the core idea still applies to our project.

2. Kovach et al. (1999)

Title: Human Resource Information Systems (HRIS): Providing Business with Rapid Data Access, Information Exchange and Strategic Advantage

Kovach and Cathcart focused on how computerized HR systems make data access faster and decision-making easier. They pointed out that manual systems are slow and error-prone, especially when it comes to salary-related records. This matched exactly with the problem we were trying to solve. Their work gave us a clearer understanding of why shifting from manual to digital payroll isn't just a luxury, it's actually necessary for any growing organization.

3. Kavanagh et al. (2017)

Title: Human Resource Information Systems: Basics, Applications, and Future Directions

This one was more like a textbook than a research paper, and it was honestly the most helpful for understanding how payroll systems are actually structured. Kavanagh explained different modules of HRIS employee database, salary calculation, tax handling, report generation — which is very similar to what we planned for our own project. It also talked about data privacy and security, which made us realize why even a small system should be careful about who can access what information.

4. Laudon et al. (2020)
Title: Management Information Systems: Managing the Digital Firm

This book doesn't specifically talk about payroll, but it covers broader concepts like databases, automation, and how information systems support business operations. Since our payroll system is essentially a small management information system, this helped us understand the bigger picture of what we were building and how it fits into real-world organizational software.

5. Hendrickson (2003)
Title: Human Resource Information Systems: Backbone Technology of Contemporary Human Resources

Hendrickson's paper describes how HRIS has become a core part of modern HR departments rather than just an optional tool. He talks about how automated payroll systems reduce paperwork, speed up salary processing, and improve transparency. What we found interesting was his emphasis on employee satisfaction — when salaries are calculated correctly and on time, it directly affects how employees feel about their organization. That's a simple but important point that we kept in mind while thinking about the purpose of our project.

Problem Formulation

In many small organizations and institutions, payroll management is still handled manually using spreadsheets or even handwritten records. At first, this may seem manageable, especially when the number of employees is limited. However, as the organization grows, handling salary calculations, attendance records, tax deductions, and allowances becomes complicated and stressful. Even a small calculation mistake can create confusion and dissatisfaction among employees. We realized that this is actually a common real-world problem, not something imaginary from textbooks.

The main problem we are trying to address through this project is the lack of an organized, efficient, and error-free system for managing payroll activities. Manual payroll processes consume time, require repeated verification, and sometimes lead to data redundancy or loss of records. Moreover, maintaining proper documentation for audits and compliance becomes difficult without a structured system. This creates unnecessary pressure on management as well as HR departments.

We, as a team, decided to work on the Payroll Management System because it felt practical and relatable. Instead of choosing a project that only sounds technically advanced, we wanted to build something that actually solves a basic but important problem. Every organization needs payroll management, whether it is a school, company, or startup. So we thought, why not design a system that simplifies this process?

Also, from a learning perspective, this project helps us understand database management, system design, and real-world application of programming concepts. We are not just building a system for marks, but trying to understand how software solutions can reduce human effort and improve efficiency. In simple words, we are working on this project because payroll management is a common problem, and we believe automation is the most logical and practical solution for it

Objectives

- To develop a menu-driven payroll system that even a beginner can understand and use.
- To automate basic salary calculation including allowances, deductions, and tax.
- To generate payslips for employees without manual effort.
- To maintain a salary history so past records aren't lost.
- To reduce the kind of manual errors that commonly happen in spreadsheet-based payroll.
- To learn and apply shell scripting concepts in a practical, real-world scenario.

Methodology

1. Steps Involved

The development of this system followed a structured, step-by-step approach:

Step 1: Requirement Identification Before writing any code, we first identified what the system needs to do. The core features we finalized are:

- Store employee details
- Record attendance
- Calculate salary
- Deduct taxes and other contributions
- Generate payslips
- Maintain salary history

Step 2: Program Structure Design We decided to use a simple menu-driven approach. The program displays a menu, the user selects an option, and the corresponding function executes. This structure is easy to implement and user-friendly.

Step 3: Module Development The program was divided into separate functions — one for each feature. This made development and testing easier, as each module could be checked individually.

Step 4: Logic Implementation Salary calculation was based on the following formulas:

- Gross Salary = Basic Salary + Allowances
- Net Salary = Gross Salary – Deductions

Attendance is also considered, as unpaid leaves affect the number of working days and the final salary.

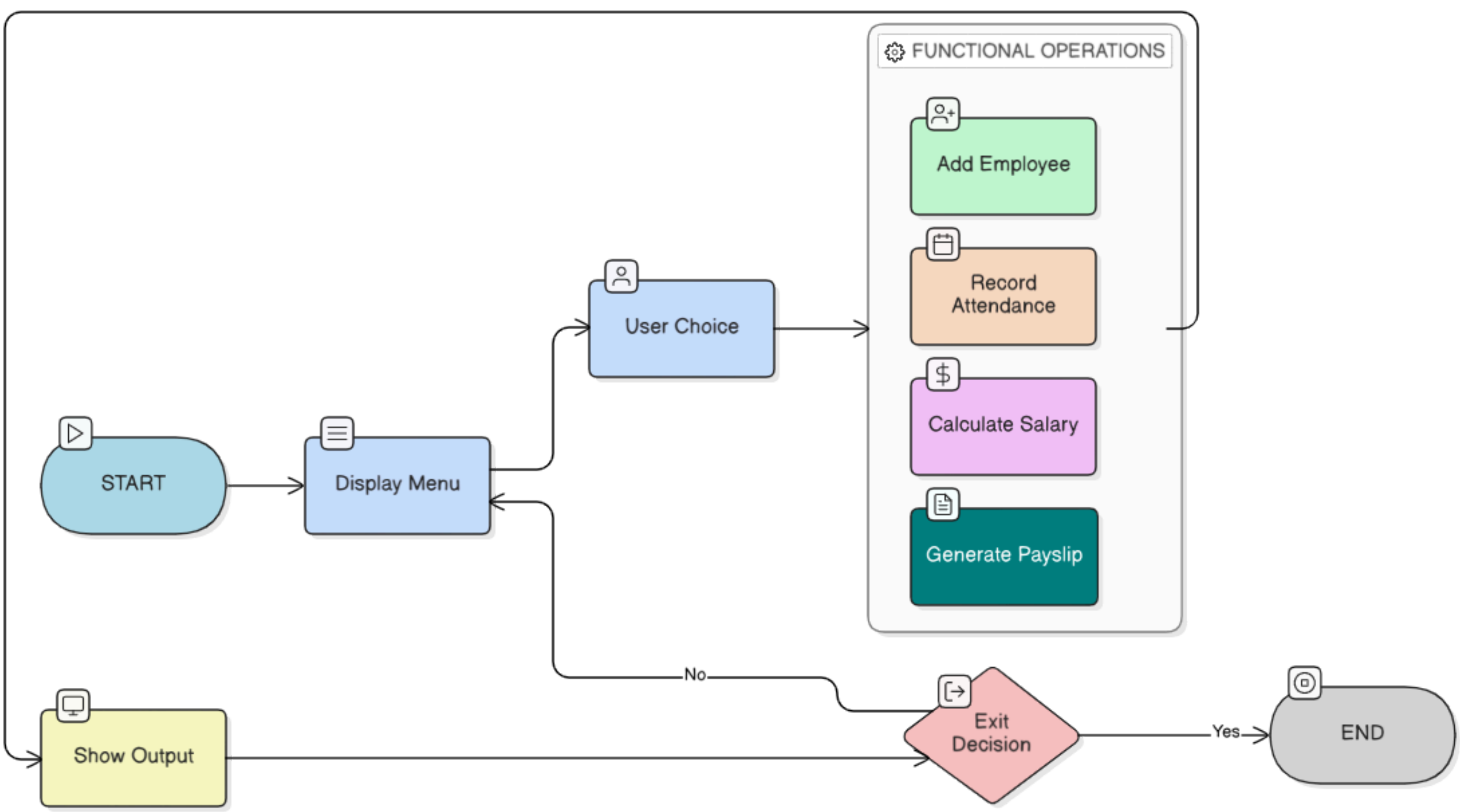
Step 5: Testing The program was tested with different inputs and results were cross-verified manually to ensure accuracy.

2. Flow of Work

The overall flow of the system is simple and logical:
Start
→ Display Menu
→ User selects option
→ Execute selected function
→ Show result
→ Return to menu
→ Exit (if chosen)

No complications. Straightforward working.

3. Basic Flow Diagram



Hardware and Software Required

Hardware and Software Requirements

1. Hardware Requirements

The project does not require any special or high-end hardware. Since it is a simple menu-driven command-line application, the basic requirement is:

- A Computer System with standard configuration
- Keyboard and Monitor

That is sufficient to develop and execute the program.

2. Software Requirements

The following software tools are required for developing and running the system:

- Operating System: Linux
- Programming Language: Bash / Shell Scripting
- Terminal: Bash Shell for execution
- Editor: VI Editor
- Permission Tool: chmod command (to make the script executable)

The system is developed as a shell script and executed directly in the Linux terminal environment.

References

1. Beadles, N. A., Lowery, C. M., & Johns, K. (2005). The Impact of Human Resource Information Systems: An Exploratory Study in the Public Sector. Communications of the IIMA.
   Available at: https://scholarworks.lib.csusb.edu/ciima/vol5/iss4/6/
2. Kovach, K. A., & Cathcart, C. E. (1999). Human Resource Information Systems (HRIS): Providing Business with Rapid Data Access, Information Exchange and Strategic Advantage. Public Personnel Management.
   Available at: https://doi.org/10.1177/009102609902800208
3. Kavanagh, M. J., & Johnson, R. D. (2017). Human Resource Information Systems: Basics, Applications, and Future Directions (3rd ed.). Sage Publications.
   Available at: https://doi.org/10.4135/9781544398525
4. Laudon, K. C., & Laudon, J. P. (2020). Management Information Systems: Managing the Digital Firm (16th ed.). Pearson Education.
   Available at:
   https://www.pearson.com/en-us/subject-catalog/p/management-information-systems-managing-the-digital-firm/P200000003479
5. Hendrickson, A. R. (2003). Human Resource Information Systems: Backbone Technology of Contemporary Human Resources. Journal of Labor Research.
   Available at: https://link.springer.com/article/10.1007/s12122-003-1002-5

Additional Technical and Conceptual References

1. The GNU Project. Bash Reference Manual.
   Available at: https://www.gnu.org/software/bash/manual/bash.html
2. Linux Documentation Project. Advanced Bash-Scripting Guide.
   Available at: https://tldp.org/LDP/abs/html/
3. IBM. What is Payroll Management?
   Available at: https://www.ibm.com/topics/payroll-management

4. Oracle. What is a Payroll System?
   Available at:
   https://www.oracle.com/human-capital-management/payroll/what-is-payroll/
5. Investopedia. Payroll: Definition, How It Works, and Components.
   Available at: https://www.investopedia.com/terms/p/payroll.asp